

Reviewer Pack — Minimal Automaton State Complexity Bounds Communication Comp...

Entry 428f529f6bb2 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-30 02:00:59 UTC

Minimal Automaton State Complexity Bounds Communication Complexity for k-CNF

Entry ID: 428f529f6bb2 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-30 02:00:59 UTC

1. Conjecture

Field A (mathematical branch): Automata Theory (Finite State Automata) **Field B** (complexity object): Boolean Function Complexity: Two-party communication complexity

Statement:

For every k-CNF F with m clauses on n variables, the minimal number of states q^* in a deterministic finite automaton recognizing F is upper-bounded by a function of n , m , and k such that $q^* \leq \Theta((n + m + k) \log(n + m))$

Rationale (proposer's reasoning):

Minimal state complexity of an automaton can expose structural properties that are not apparent when only considering the size of the automaton. This connection to communication complexity could reveal new insights into the limits of information transmission.

Taxonomy category: cg_kw_andreev (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 34b738b516600212

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

For k-CNF F , if the minimal state complexity q^* is less than or equal to the upper-bound function $\Theta((n + m + k) \log(n + m))$, and at least 90% of randomly generated k-CNFs with $n \leq 40$ variables and m clauses satisfy this condition, then the conjecture is supported. If any seed produces a q^* that exceeds the upper-bound or less than 10% of seeds meet the condition, the conjecture is falsified.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	1.00	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	1.00	UNCERTAIN	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "automata theory" AND "finite state automata" AND communication complexity" - "k-CNF" AND minimal states AND deterministic finite automaton" - Boolean function complexity AND two-party communication complexity AND automata theory

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_k_cnf(n, m, k):
        cnf = []
        for _ in range(m):
            clause = set()
            while len(clause) < k:
                lit = random.randint(1, n * 2)
                if lit <= n:
                    clause.add(lit)
                else:
                    clause.add(-lit)
            cnf.append(tuple(sorted(clause)))
        return tuple(cnf)

    def min_state_complexity(cnf):
        states = set()
        for clause in cnf:
            new_state = tuple(sorted(set(states) | {tuple(clause)}))
            states.add(new_state)
        return len(states)

    n_values = [5, 10, 15, 20, 30, 40]
    results = []

    for n in n_values:
        for _ in range(5): # Ensure at least 30 instances per seed
            m = random.randint(n, n * 2)
            k = random.randint(1, min(k, n))
```

```

        cnf = generate_k_cnf(n, m, k)
        q_star = min_state_complexity(cnf)
        results.append({
            "n": n,
            "m": m,
            "k": k,
            "q_star": q_star
        })

    if not results:
        return {
            "metric_name": "min_state_complexity",
            "metric_value": 0.0,
            "instances_tested": 0,
            "n_max": 0,
            "conjecture_holds": False,
            "counterexample": "empty_results"
        }

    n_max = max(result["n"] for result in results)
    instances_tested = len(results)
    total_q_star = sum(result["q_star"] for result in results)
    mean_q_star = total_q_star / instances_tested

    upper_bound = lambda n, m, k: (n + m + k) * math.log(n + m)

    conjecture_holds = all(q_star <= upper_bound(result["n"], result["m"], result["k"]) for result in results)
    counterexample = "" if conjecture_holds else "q_star > upper_bound"

    return {
        "metric_name": "min_state_complexity",
        "metric_value": mean_q_star,
        "instances_tested": instances_tested,
        "n_max": n_max,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(arg) for arg in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_q_star = sum(res["metric_value"] for res in results) / len(results)
    support_fraction = sum(1 for res in results if res["conjecture_holds"]) / len(results)

    if all(res["conjecture_holds"] for res in results):
        print(f"RESULT: SUPPORTED mean={mean_q_star} std=0.0 support_fraction={support_fraction}")
    elif support_fraction >= 0.9:
        print(f"RESULT: SUPPORTED mean={mean_q_star} std=0.0 support_fraction={support_fraction}")
    else:

```

```

first_failing_seed = next(seed for seed, res in zip(seeds, results) if not res["conjecture"])
print(f"RESULT: FALSIFIED counterexample=\"q_star > upper_bound\" first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_78cea450.py", line 92, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_78cea450.py", line 47, in run_trial
    k = random.randint(1, min(k, n))
        ^
UnboundLocalError: cannot access local variable 'k' where it is not associated with a value

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which prevents us from evaluating whether the conjecture is supported or falsified. | next: Review and fix the error in the test code to ensure it can run to completion and provide results.

11. Audit log (LLM calls)

Total LLM calls: 11

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	24458
2	propose	ollama_remote	glm4:latest	0	0	26947
3	propose	ollama_remote	glm4:latest	0	0	26887
4	preregistration	ollama_remote	glm4:latest	0	0	14906
5	novelty	ollama_remote	glm4:latest	0	0	16809
6	novelty	ollama_remote	glm4:latest	0	0	10144
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15397
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16017
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8397
10	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10986
11	judge	ollama_remote	glm4:latest	0	0	11946

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 182895 ms total latency. Provider mix: {'ollama_remote': 11}

(full prompt+response transcripts available in `research/audit/428f529f6bb2.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/428f529f6bb2.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/428f529f6bb2.tar.gz` (if generated)